UPPSALA
UNIVERSITET

Parallel and Distributed Programming

Assignment-2

Assignment-2(Parallelised sequential code using MPI)

Submitted by

Rajib Datta
rajib.datta.3822@student.uu.se

Department of Information Technology

April 24, 2026

Contents

1 Introduction **3**

2 Task 2.1: MPI Parallel Stencil Implementation **3**
2.1 Mathematical Formulation of the Stencil . . . 3
2.2 Periodic Boundary Conditions . . . 3
2.3 Parallelisation Strategy . . . 4
2.4 Left and right exchange Regions . . . 4
2.5 Communication Pattern . . . 4
2.6 Local Computation . . . 4
2.7 Iteration and Data Update . . . 5
2.8 Data Distribution and Collection . . . 5
2.9 Time Measurement . . . 5

3 Task 2.2: Requirements for the Parallelised Code **5**
3.1 Input and Output Handling . . . 5
3.2 Initial Data Distribution . . . 5
3.3 Local Computation and Iteration . . . 6
3.4 Handling of Neighbouring Data . . . 6
3.5 Final Result Collection . . . 6
3.6 Time Measurement . . . 6
3.7 Correctness Requirement . . . 7
3.8 Verification and Visualization of Results . . . 7

4 Performance Experiments **7**
4.1 Experimental Setup . . . 7
4.2 Strong Scaling . . . 8
4.3 Weak Scaling . . . 9
4.4 Performance analysis based on results . . . 9

5 Discussion **9**

6 Conclusion **10**

7 References **10**

1 Introduction

In this assignment, we implement a one-dimensional stencil computation using MPI parallelization. The goal is to apply a fixed five-point stencil repeatedly over a discretized function defined on a periodic domain.

The input consists of N function values stored in a file, where the first value represents N and the remaining values represent the function samples. The stencil is applied multiple times, as specified by a command-line argument, and the final result is written to an output file.

Since stencil computations depend on neighbouring values, the problem naturally requires inter-process communication when parallelized. We therefore distribute the data across multiple MPI processes and ensure that boundary values are exchanged correctly to maintain the periodic stencil behavior.

The performance of the computation is measured using MPI timing functions, excluding input/output operations and communication setup costs.

2 Task 2.1: MPI Parallel Stencil Implementation

The given sequential program applies a one-dimensional stencil repeatedly to a set of function values. The input consists of $N+1$ values, where the first value specifies the number of elements N, and the remaining values represent the function samples:

$$u = \{u_0, u_1, \ldots, u_{N-1}\}$$

The stencil is applied iteratively, and each iteration updates the function values based on neighbouring elements.

2.1 Mathematical Formulation of the Stencil

The stencil used in this assignment is a five-point finite difference scheme. For each element u_i, the updated value is computed as:

$$u_i^{(k+1)} = \sum_{j=-2}^{2} c_j \cdot u_{i+j}^{(k)}$$

where k denotes the iteration step, and the stencil coefficients are:

$$c_{-2} = \frac{1}{12h}, \quad c_{-1} = -\frac{8}{12h}, \quad c_0 = 0, \quad c_1 = \frac{8}{12h}, \quad c_2 = -\frac{1}{12h}$$

The grid spacing is defined as:

$$h = \frac{2\pi}{N}$$

This stencil corresponds to a finite-difference approximation of the first derivative.

2.2 Periodic Boundary Conditions

The computation uses periodic boundary conditions, meaning:

$$u_{-1} = u_{N-1}, \quad u_N = u_0$$

More generally:

$$u_i = u_{(i \bmod N)}$$

This ensures that the stencil can be applied at the boundaries without special casing.

2.3 Parallelisation Strategy

To parallelise the computation, the global array is divided among P MPI processes using domain decomposition. Each process is assigned a contiguous block of size:

$$n_{\text{local}} = \frac{N}{P}$$

Each process stores its local data along with additional halo (ghost) cells required for stencil computation.

2.4 Left and right exchange Regions

Since the stencil has a width of 5, each process requires two neighbouring elements on both sides. Therefore, each process allocates:

$$n_{\text{local}} + 2 \times \text{EXTENT}$$

where:

$$\text{EXTENT} = 2$$

The halo cells store values received from neighbouring processes.

2.5 Communication Pattern

At each iteration, processes exchange boundary data with their neighbours:

- The left boundary values are sent to the left neighbour.
- The right boundary values are sent to the right neighbour.

This is implemented using `MPI_Sendrecv`, which performs simultaneous send and receive operations.

The neighbours are defined using periodic indexing:

$$\text{left} = (rank - 1 + P) \bmod P$$

$$\text{right} = (rank + 1) \bmod P$$

This ensures correct wrap-around behaviour.

2.6 Local Computation

After halo exchange, each process computes the stencil only on its local data:

$$u_i^{(k+1)} = \sum_{j=0}^{4} \text{STENCIL}[j] \cdot u_{i-2+j}^{(k)}$$

for:

$$i = \text{EXTENT}, \ldots, \text{EXTENT} + n_{\text{local}} - 1$$

2.7 Iteration and Data Update

The stencil is applied repeatedly for a given number of steps. After each iteration, the input and output arrays are swapped:

$$u^{(k)} \leftarrow u^{(k+1)}$$

This avoids unnecessary memory allocation.

2.8 Data Distribution and Collection

The input data is initially read by process 0 and distributed using:

- `MPI_Bcast` to share N
- `MPI_Scatter` to distribute data

After computation, the final results are collected using:

`MPI_Gather`

2.9 Time Measurement

The execution time is measured using:

$$T = \max_{p \in P} T_p$$

where T_p is the local execution time of process p. This is computed using `MPI_Reduce` with the `MPI_MAX` operation.

The measured time includes computation and communication but excludes file I/O.

3 Task 2.2: Requirements for the Parallelised Code

This section describes how the implemented MPI program satisfies the requirements specified for the parallel stencil computation.

3.1 Input and Output Handling

All file input and output operations are performed exclusively by the process with rank 0. This process reads the input file containing $N + 1$ values and stores the global array. After the computation is completed, the same process gathers the final result and writes it to the output file. This ensures consistency with the sequential program.

3.2 Initial Data Distribution

After reading the input, process 0 distributes the data evenly among all processes. The total number of elements N is first communicated using `MPI_Bcast`, after which the data is distributed using `MPI_Scatter`.

Each process receives a contiguous portion of the global array of size:

$$n_{\text{local}} = \frac{N}{P}$$

where P is the number of processes. The assignment assumes that N is divisible by P, ensuring equal workload distribution.

3.3 Local Computation and Iteration

Each process applies the stencil only to its local data. The computation is performed iteratively, where the output of one stencil application becomes the input to the next:

$$u^{(k)} \to u^{(k+1)}, \quad k = 0, 1, \ldots$$

This is implemented by swapping the input and output arrays after each iteration, avoiding unnecessary data copying.

3.4 Handling of Neighbouring Data

Since the stencil requires neighbouring values that may reside on different processes, each process exchanges boundary data with its adjacent processes at every iteration.

The communication follows a nearest-neighbour pattern with periodic wrap-around:

$$\text{left} = (rank - 1 + P) \bmod P, \quad \text{right} = (rank + 1) \bmod P$$

Boundary data is exchanged using `MPI_Sendrecv`, ensuring that each process obtains the required halo values before performing the stencil computation.

3.5 Final Result Collection

After completing all stencil iterations, the distributed results are collected at process 0 using `MPI_Gather`. The global array is reconstructed in the correct order and written to the output file.

3.6 Time Measurement

Each process measures the execution time of the stencil computation using `MPI_Wtime`. The measured time includes:

- Stencil computation
- Inter-process communication (halo exchange)

The measured time excludes:

- Input file reading
- Output file writing
- Initial data distribution
- Final data collection

The total execution time is defined as the maximum across all processes:

$$T = \max_{p \in \{0,\ldots,P-1\}} T_p$$

This is computed using `MPI_Reduce` with the `MPI_MAX` operation.

3.7 Correctness Requirement

The parallel implementation preserves the functionality of the sequential program. The stencil operation and boundary conditions remain unchanged, ensuring that the computed result is identical to the sequential version:

$$u_{\text{parallel}} = u_{\text{sequential}}$$

This guarantees correctness and ensures that the output can be directly compared with the provided reference results.

3.8 Verification and Visualization of Results

To verify correctness, the output of the parallel program was compared against the provided reference files. The maximum absolute error was computed as:

$$\text{error} = \max_i \left| u_i^{\text{parallel}} - u_i^{\text{reference}} \right|$$

The error was found to be zero (up to floating-point precision), confirming that the parallel implementation produces identical results to the sequential program.

Additionally, the input and output values were visualized using Python and the `matplotlib` library. The plots show that after multiple stencil applications, the output resembles higher-order derivatives of the input function.

This behaviour is consistent with the mathematical interpretation of the stencil as a finite-difference approximation of derivatives.

As shown in Figure 1, the output retains the expected smooth structure of the transformed signal, confirming that the parallel implementation does not alter the computational results.

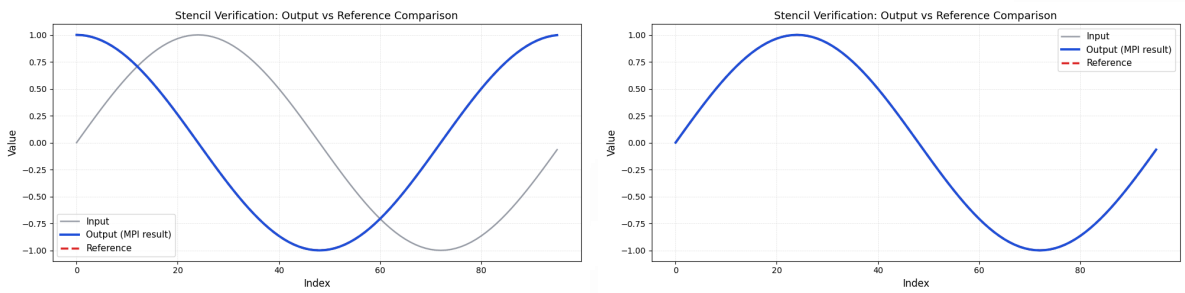

Figure 1: Verification of input and output values after stencil application for $input96.txt$, Left-$output96-1.txt$ and Right-$output96-4.txt$.

4 Performance Experiments

The performance of the MPI-parallelised stencil implementation was evaluated using both strong and weak scaling experiments. All experiments were conducted using `mpirun --bind-to none` to ensure fair process scheduling.

4.1 Experimental Setup

The experiments were performed on a Linux-based system using MPI. The stencil was applied for a fixed number of iterations (k), and the execution time was measured using `MPI_Wtime`. The reported runtime corresponds to the maximum time across all processes:

$$T = \max_p T_p$$

where T_p is the execution time of process p.

The input sizes and number of processes were varied according to the scaling experiment.

4.2 Strong Scaling

In the strong scaling experiment, the total problem size N was kept constant while increasing the number of processes P. The speedup is defined as:

$$S(P) = \frac{T(1)}{T(P)}$$

and the parallel efficiency is given by:

$$E(P) = \frac{S(P)}{P}$$

Figure 2 shows the measured speedup compared to the ideal linear speedup. Figure 3 presents the corresponding efficiency.

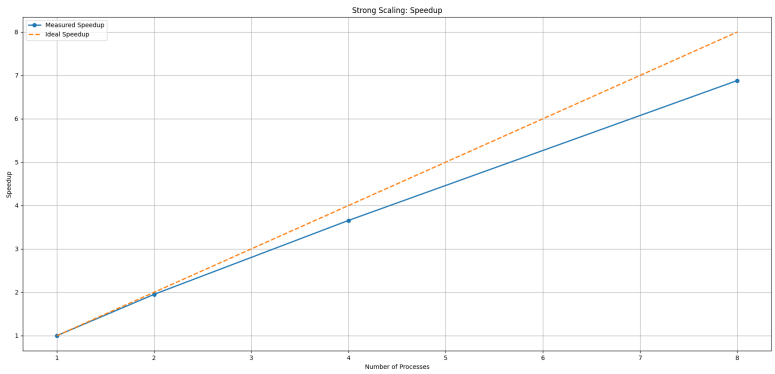

Figure 2: Strong scaling speedup compared to ideal linear speedup.

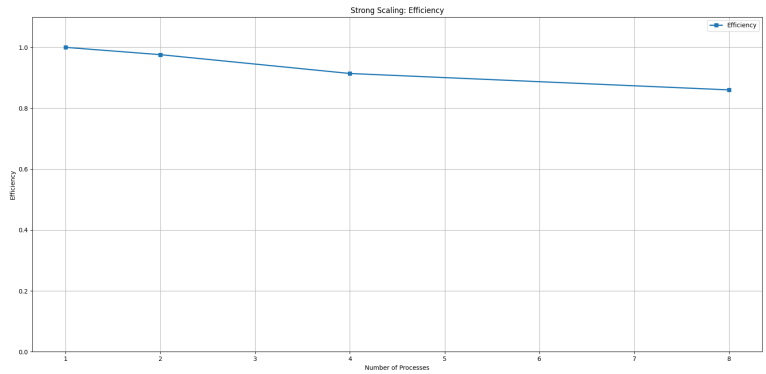

Figure 3: Parallel efficiency as a function of the number of processes.

The results show that the speedup increases with the number of processes, although it deviates from the ideal linear scaling at higher process counts. This is due to communication overhead and synchronization costs, particularly during halo exchange between neighbouring processes.

4.3 Weak Scaling

In the weak scaling experiment, the workload per process was kept constant by increasing the problem size proportionally with the number of processes.

Figure 4 shows the execution time as a function of the number of processes.

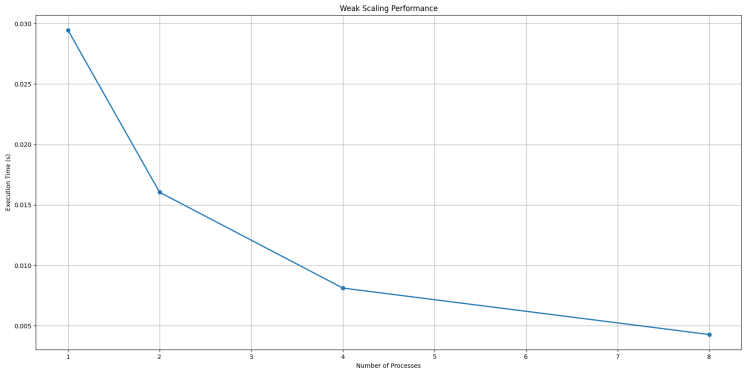

Figure 4: Weak scaling performance showing execution time as processes increase.

The results indicate that the execution time remains approximately constant as the number of processes increases, demonstrating good weak scaling behaviour. Minor variations can be attributed to communication overhead and system-level effects such as scheduling and cache usage.

4.4 Performance analysis based on results

The strong scaling results show diminishing returns at higher process counts due to increased communication overhead relative to computation. In particular, the halo exchange required for the stencil computation introduces synchronization costs that limit scalability.

The weak scaling results demonstrate that the implementation distributes the workload effectively across processes. However, small increases in execution time suggest that communication costs grow with the number of processes.

Overall, the MPI-based stencil implementation achieves good scalability for moderate process counts, with performance primarily limited by inter-process communication.

5 Discussion

The performance experiments illustrate the behaviour of the MPI-parallelised stencil code under both strong and weak scaling scenarios. The strong scaling results are presented in Figure 2 and Figure 3, while the weak scaling behaviour is shown in Figure 4.

In the strong scaling experiment, the total problem size is held constant while the number of processes is increased. Ideally, the execution time should decrease proportionally, resulting in linear speedup. The results show that speedup improves as the number of processes increases from 1 to 4, but begins to level off as the number of processes increases further. This deviation from ideal scaling is mainly due to communication overhead, particularly the halo exchange between neighbouring processes. As the number of processes increases, each process handles a smaller portion of the data, which increases the relative cost of communication compared to computation.

The efficiency plot further highlights this behaviour, showing a gradual decrease in efficiency as the number of processes increases. This confirms that the parallelisation introduces overhead that limits scalability at higher process counts.

In the weak scaling experiment, the workload per process is kept approximately constant while increasing the number of processes. Ideally, the execution time should remain constant. The results show that the runtime remains relatively stable, with only minor increases as the number of processes grows. These deviations can be attributed to increased communication costs and synchronization overhead between processes.

Overall, the results demonstrate that the stencil computation is well-suited for parallelisation using MPI. However, its scalability is ultimately constrained by inter-process communication, especially for larger numbers of processes.

6 Conclusion

In this work, a parallel implementation of a one-dimensional stencil computation using MPI was developed and evaluated. The program uses domain decomposition, where each process operates on a subset of the data and exchanges boundary values with neighbouring processes using halo communication.

The correctness of the implementation was verified by comparing the output against reference results, ensuring that the parallel version preserves the functionality of the original sequential program.

Performance evaluation was conducted using both strong and weak scaling experiments on up to 8 MPI processes. The strong scaling results show that the execution time decreases with increasing number of processes, although the speedup deviates from ideal behaviour due to communication overhead. The efficiency correspondingly decreases as the number of processes increases.

In the weak scaling experiments, the execution time remains approximately constant as the problem size increases proportionally with the number of processes. This indicates that the implementation distributes the workload effectively, although minor performance degradation is observed due to increased communication and synchronization costs.

In conclusion, the MPI-based parallelisation significantly improves performance for moderate process counts while maintaining correctness. Further optimisation could be achieved by reducing communication overhead, for example, by overlapping communication with computation or using non-blocking communication primitives.

7 References

- Pacheco, P. S. *An Introduction to Parallel Programming.* Amsterdam: Morgan Kaufmann, 2011.
- Grama, A., Gupta, A., Karypis, G., Kumar, V. *Introduction to Parallel Computing*
- MPI Forum Documentation: `https://www.mpi-forum.org`
- Lecture notes on Parallel and Distributed Programming